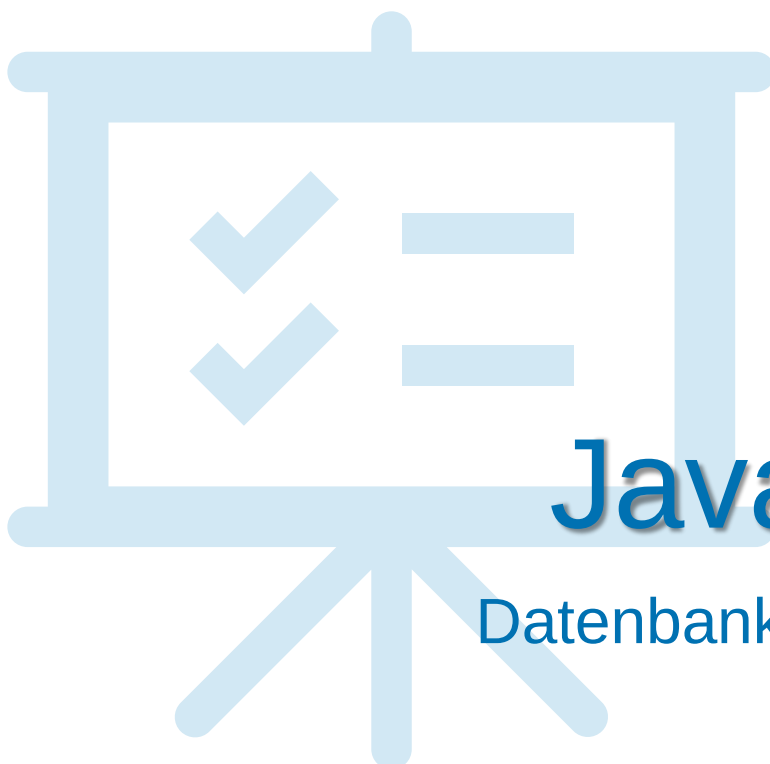




Java Programmierung

Datenbankverbindung mit **IntelliJ** und **MySQL**



Themenübersicht

01

XAMPP

- 1. Download und Installation

02

**MySQL Connector
und IntelliJ**

- 2.1 Download
- 2.2 Einrichten der Entwicklungsumgebung

03

**PHP My Admin
und IntelliJ**

- 3.1 Datenbank in PHP My Admin erstellen
- 3.2 Datenbankabfrage in Java



01

XAMPP

Anwendung zum Betreiben eines Webserver mit MySQL und PHP My Admin ←

Diesen Schritt können Sie überspringen, wenn Sie bereits XAMPP installiert haben



 **XAMPP** Apache + MariaDB + PHP + Perl

Was ist XAMPP?

XAMPP ist die beliebteste PHP-Entwicklungsumgebung

XAMPP ist eine vollständig kostenlose, leicht zu installierende Apache-Distribution, die MariaDB, PHP und Perl enthält. Das XAMPP Open-Source-Paket wurde für eine extrem einfache Installation und Nutzung eingerichtet.


XAMPP

Herunterladen
Klicken Sie hier für weitere Versionen

 **XAMPP für Windows**
8.0.10 (PHP 8.0.10)

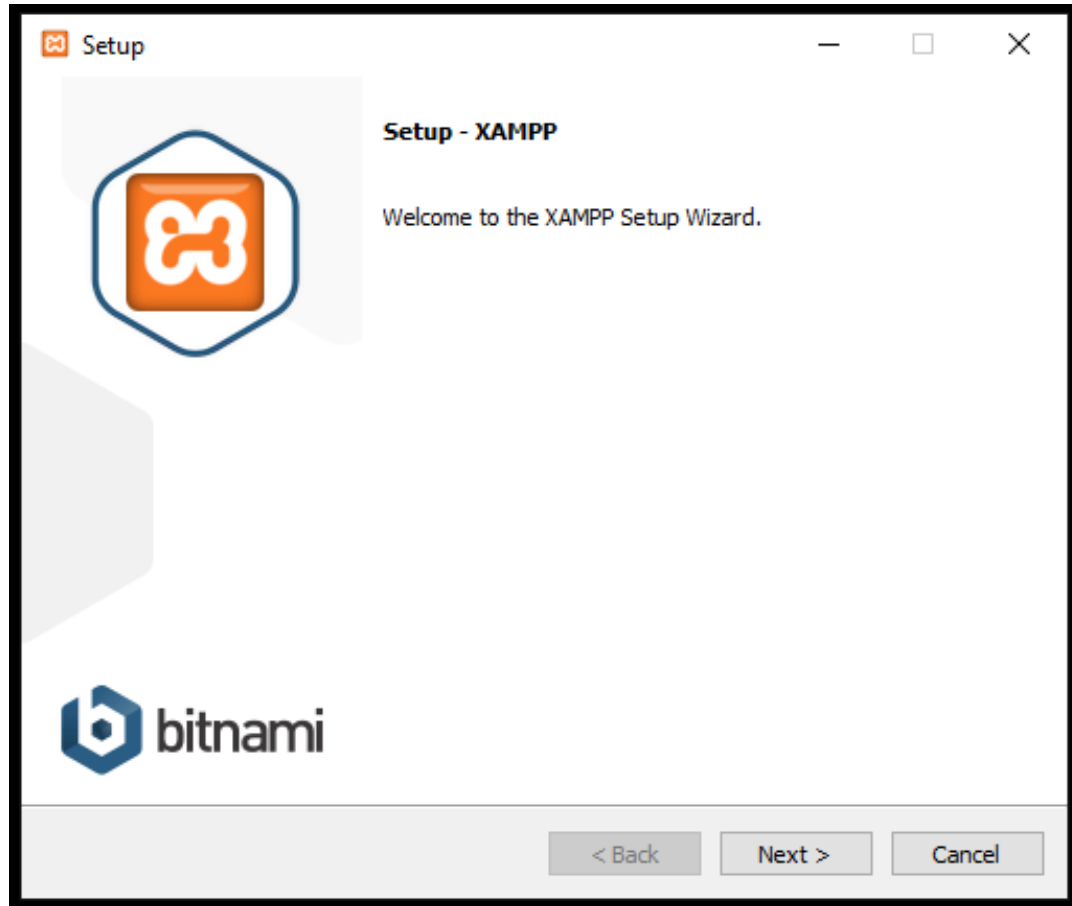
 **XAMPP für Linux**
8.0.10 (PHP 8.0.10)

 **XAMPP für OS X**
8.0.10 (PHP 8.0.10)

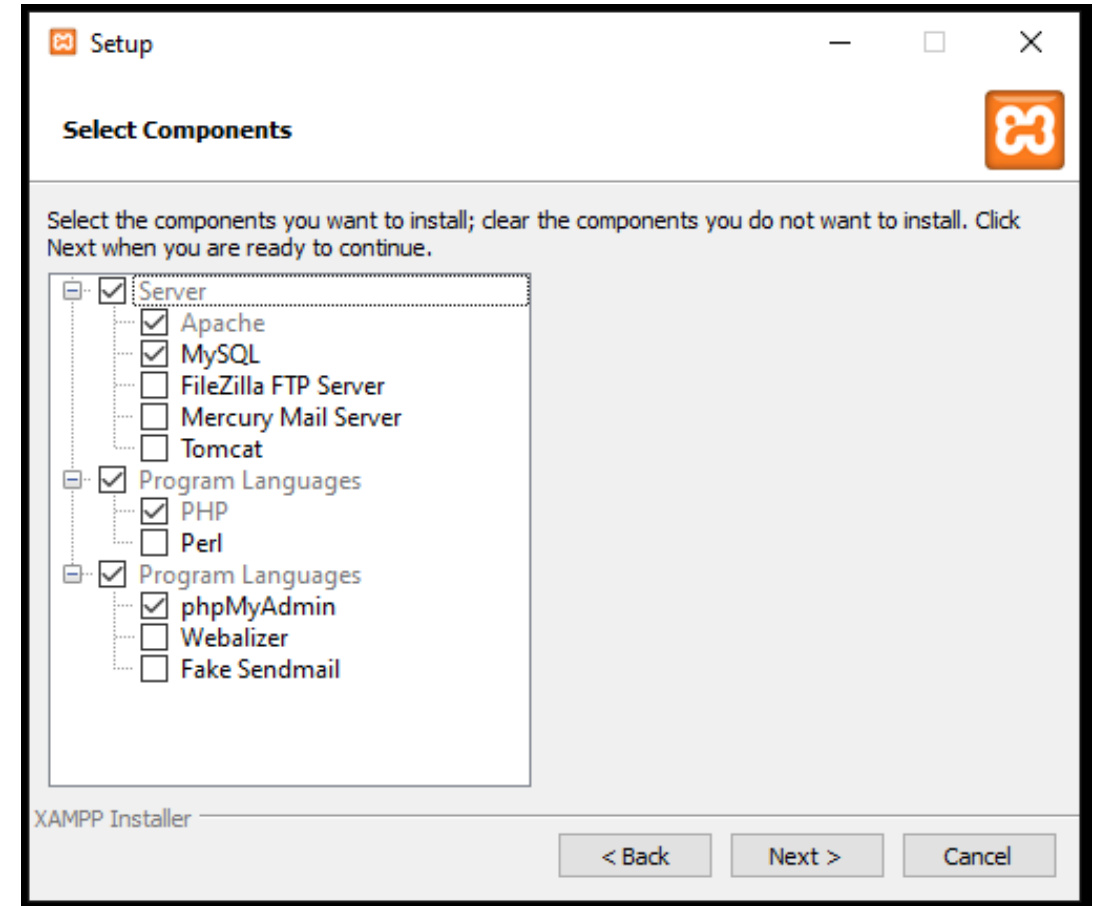
Unter folgendem Link können Sie XAMPP herunterladen.

<https://www.apachefriends.org/de/index.html>

Installation XAMPP

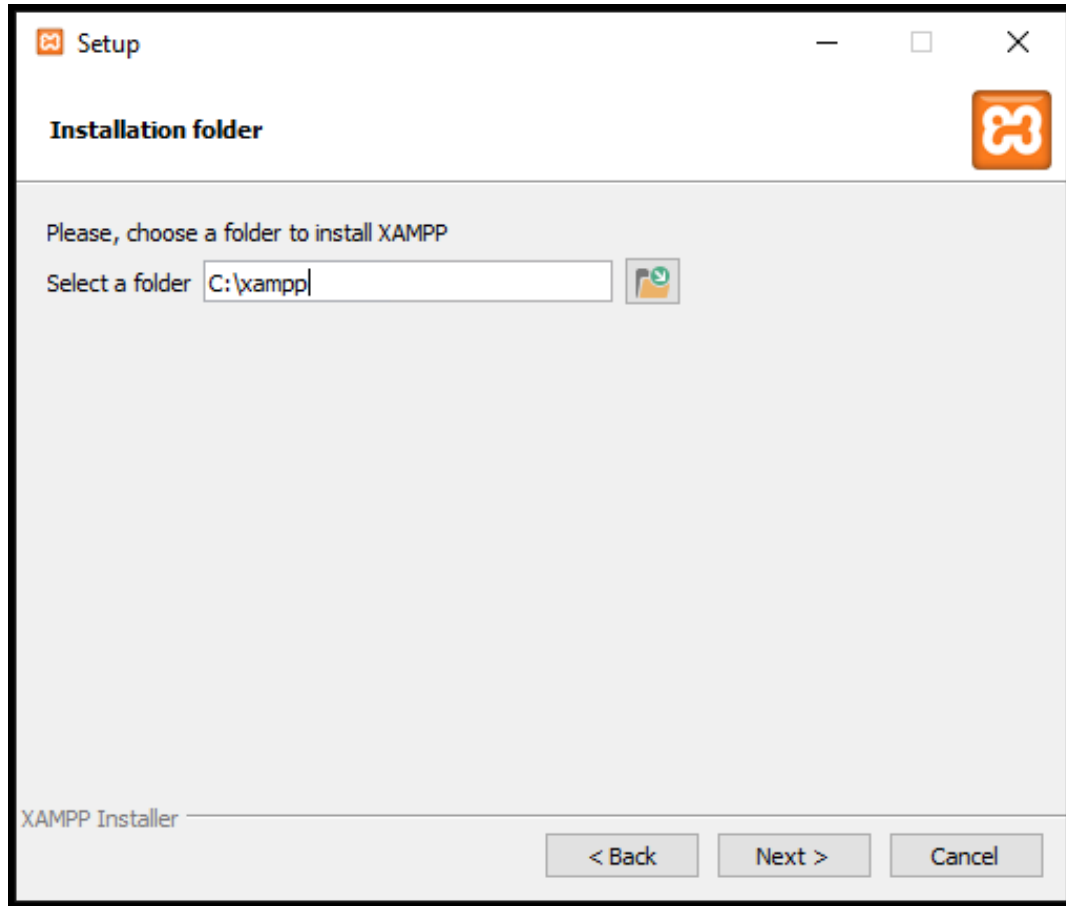


Starten Sie die Installation durch Doppelklick auf die heruntergeladene Setup Anwendung

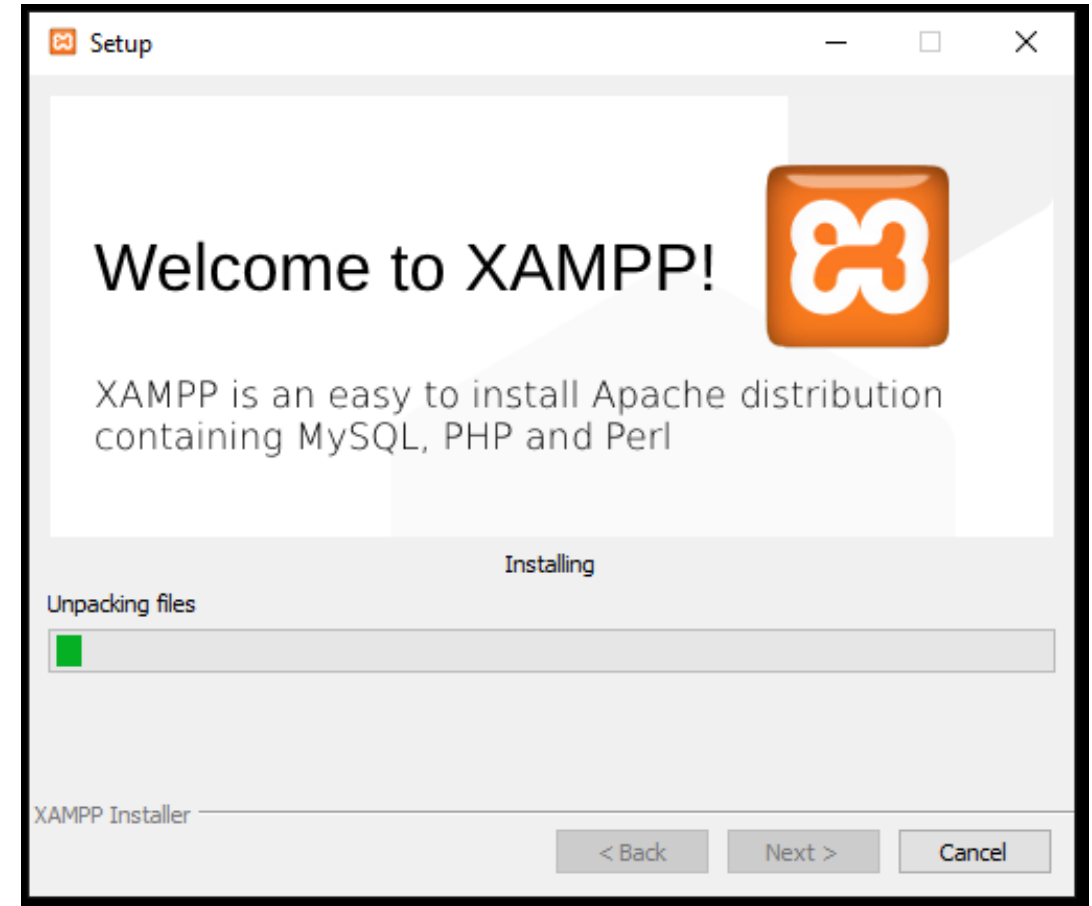


Die Installation verläuft wie jede andere und Sie können die Einstellungen auf ihren Standard-Werten lassen. Mindestens aber brauchen wir MySQL und phpMyAdmin.

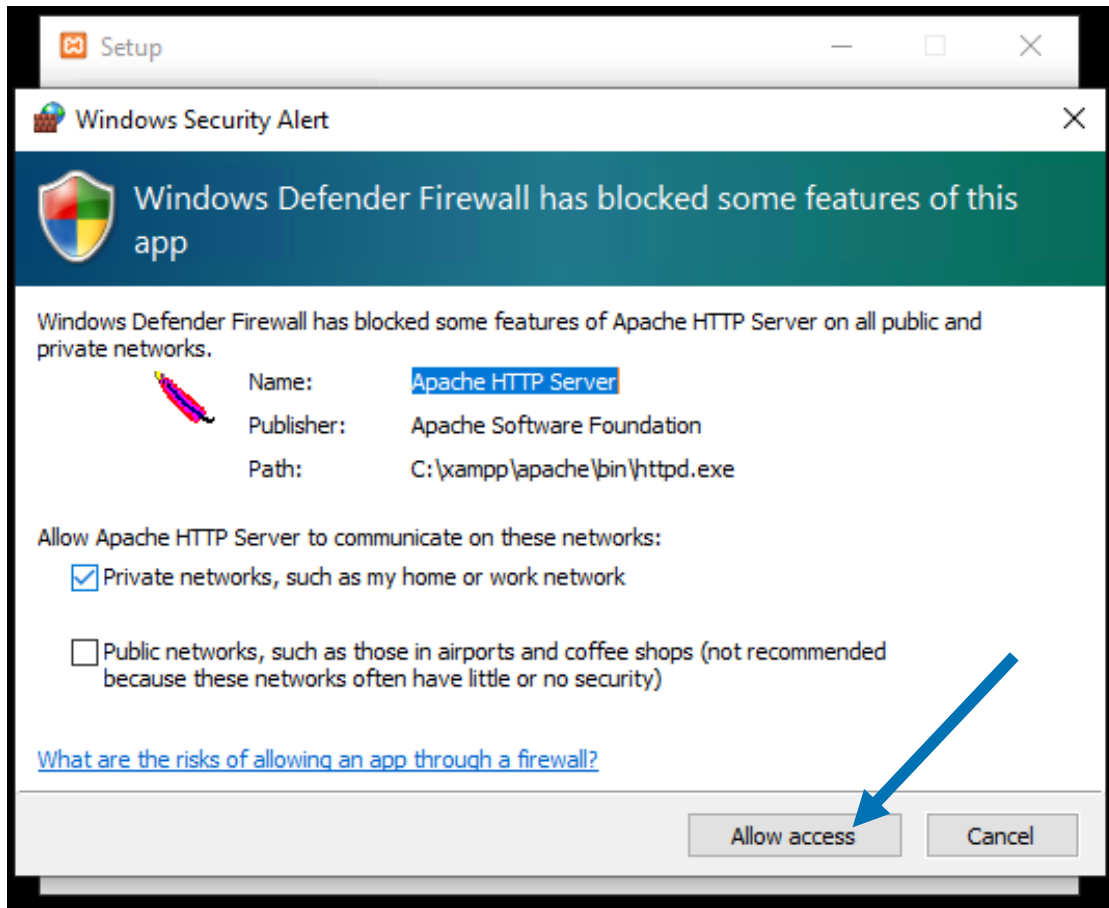
Installation XAMPP



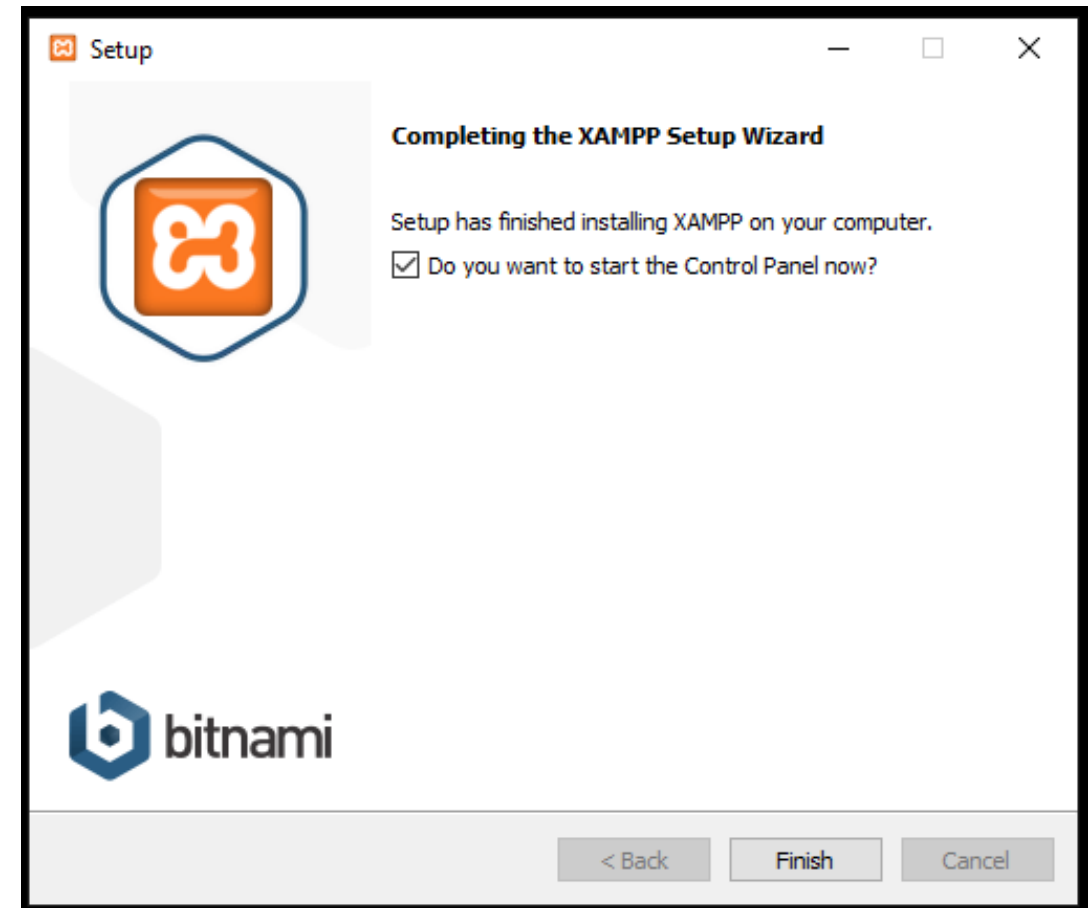
Installieren Sie XAMPP direkt unter C:\xampp



Installation XAMPP



Eventuell werden Sie darum gebeten, den Apache HTTP Server durch die Windows Firewall zu lassen. Dies können Sie bestätigen.



„Sie haben XAMPP
erfolgreich installiert.“



02

MySQL Connector und IntelliJ

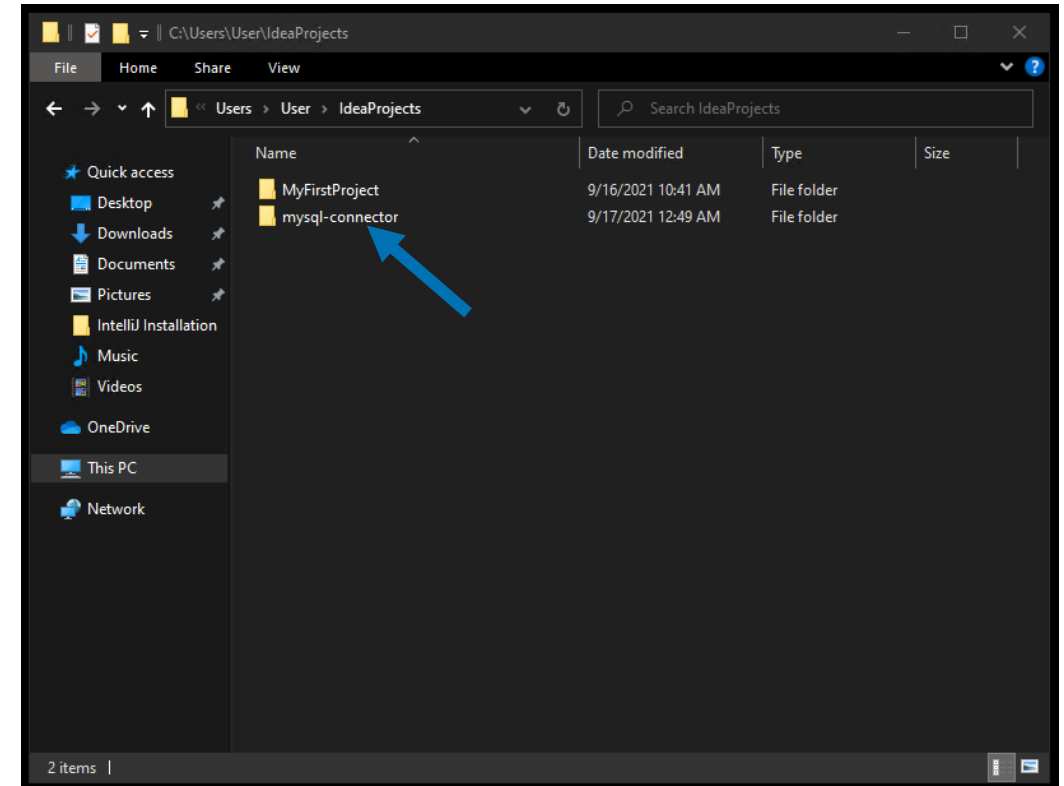
2.1 Download ←

2.2 Einrichten der Entwicklungsumgebung

Um in Java auf MySQL Datenbanken zugreifen zu können, muss ein MySQL Connector installiert werden. Auf diesen Connector werden wir in unseren Java Projekten, die Datenbanken verwenden sollen, verweisen.



Download von MySQL Connector



Unter folgendem Link können Sie den MySQL Connector herunterladen.

<https://dev.mysql.com/downloads/connector/j/>

Wählen Sie als Betriebssystem „Platform Independent“ aus und laden Sie das Zip-Archiv herunter.

Entpacken Sie das Archiv in Ihren IdeaProjects. Dadurch kann der Connector in IntelliJ leichter gefunden werden.

02

MySQL Connector und IntelliJ

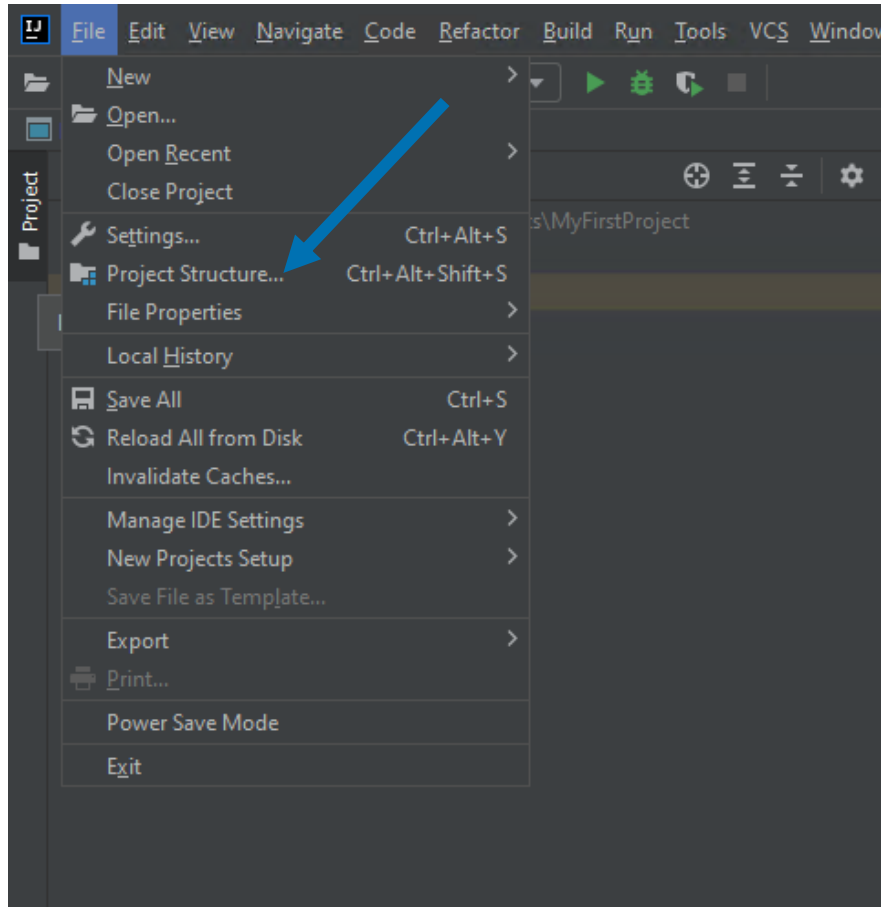
2.1 Download

2.2 Einrichten der Entwicklungsumgebung ←

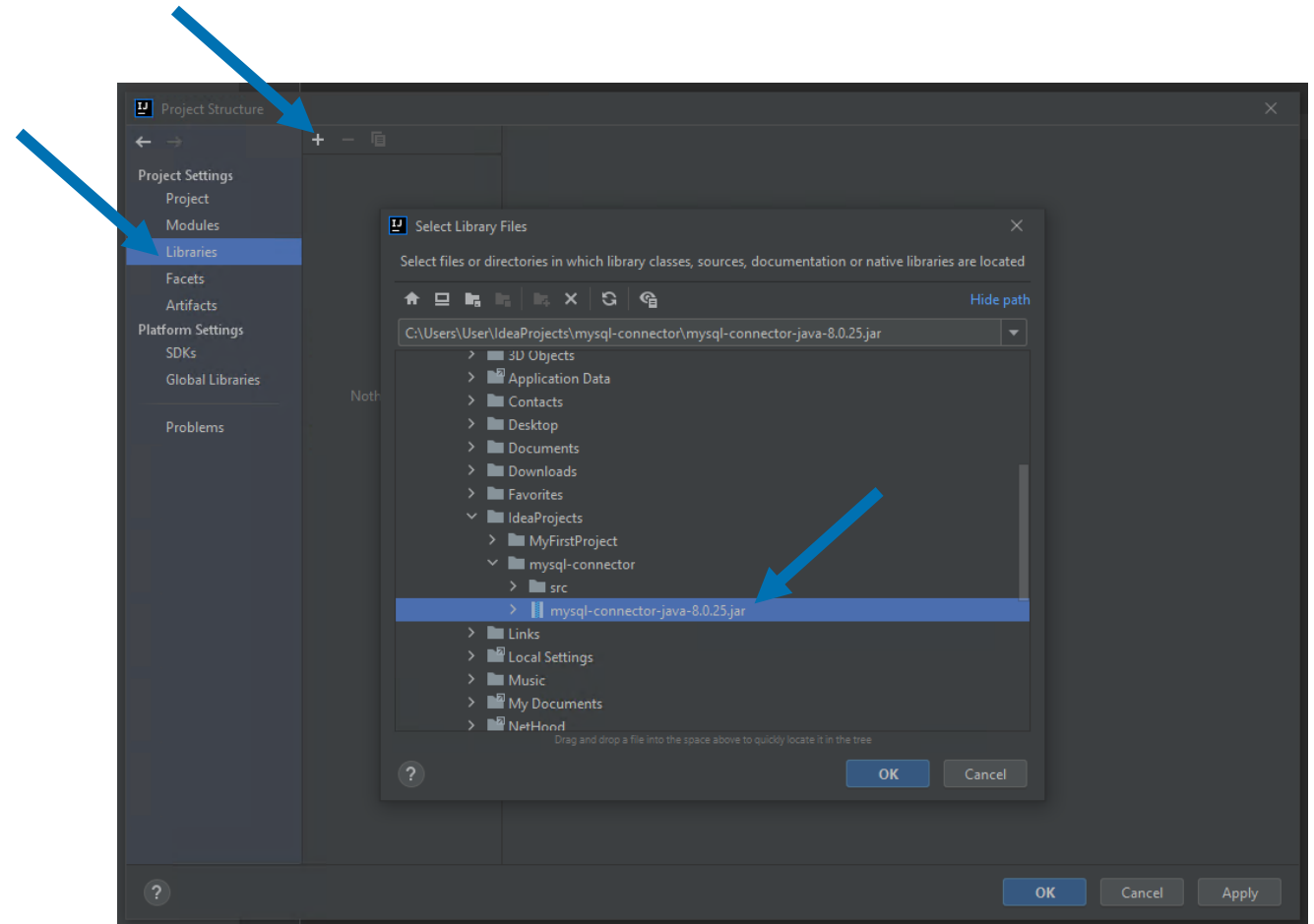
Um in Java auf MySQL Datenbanken zugreifen zu können, muss ein MySQL Connector installiert werden. Auf diesen Connector werden wir in unseren Java Projekten, die Datenbanken verwenden sollen, verweisen.



Einrichten der Entwicklungsumgebung



Starten Sie IntelliJ mit einer einfachen Konsolenanwendung
Gehen Sie in IntelliJ auf File -> Project Structure



Im neuen Fenster gehen Sie links auf Libraries, klicken auf das + Symbol, klicken auf „Java“ und wählen die mysql-connector-java-x.x.xx.jar aus

Bestätigen Sie mit OK

„Sie haben erfolgreich
den MySQL Connector
installiert.“



03

PHP My Admin und IntelliJ

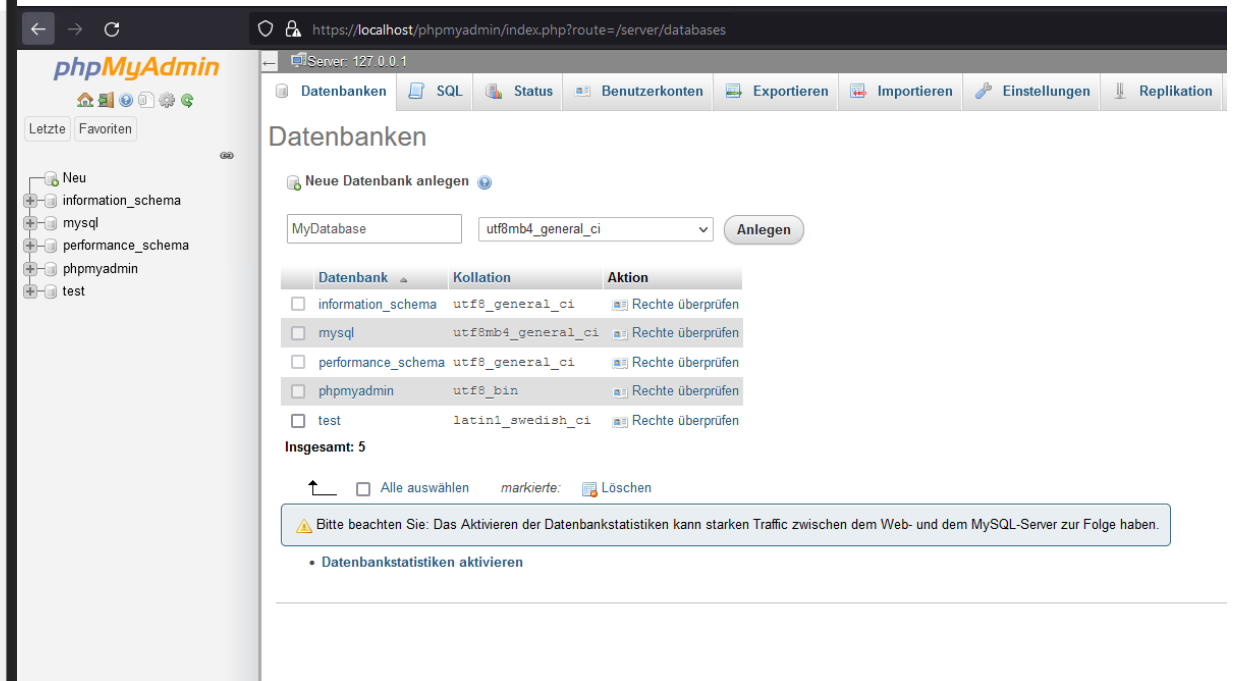
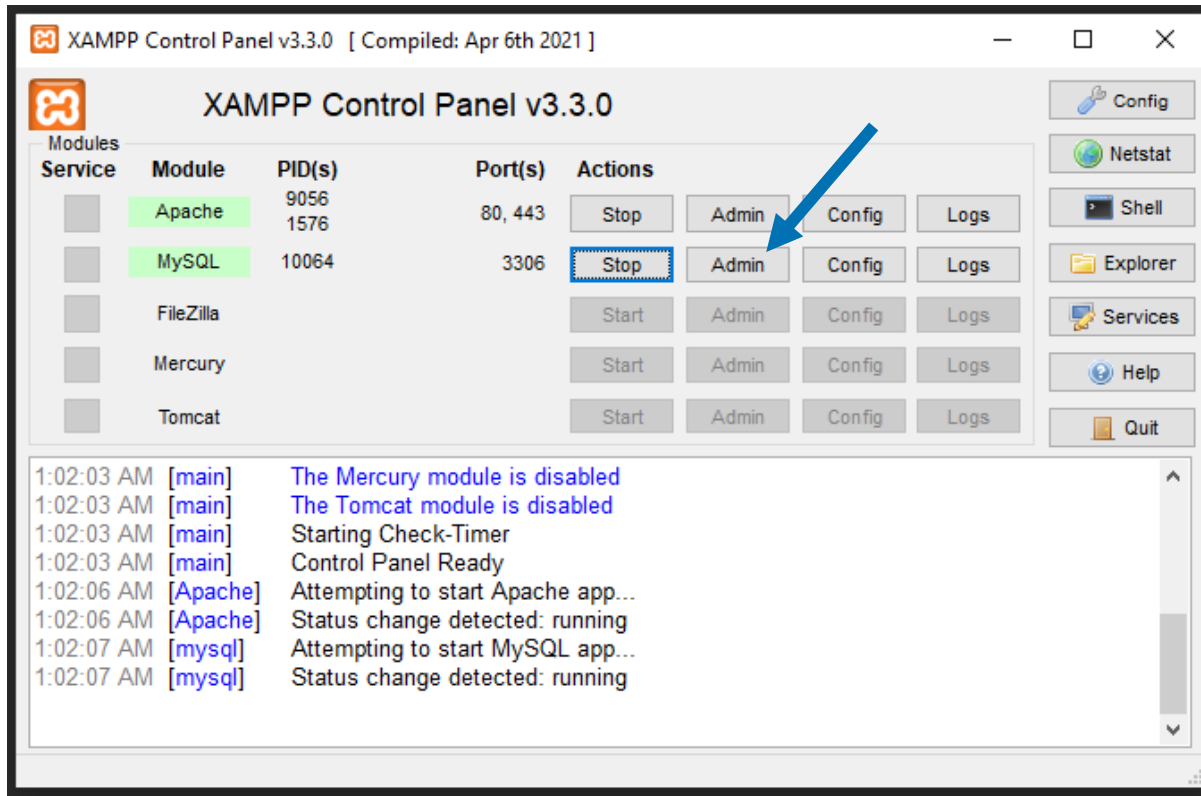
3.1 Datenbank in PHP My Admin erstellen ←

3.2 Datenbankabfrage in Java

Zuerst erstellen wir eine Testdatenbank mit einer Tabelle und Beispieldaten in PHP My Admin und fragen diese Tabelle anschließend in Java ab.



Datenbank in PHP My Admin erstellen



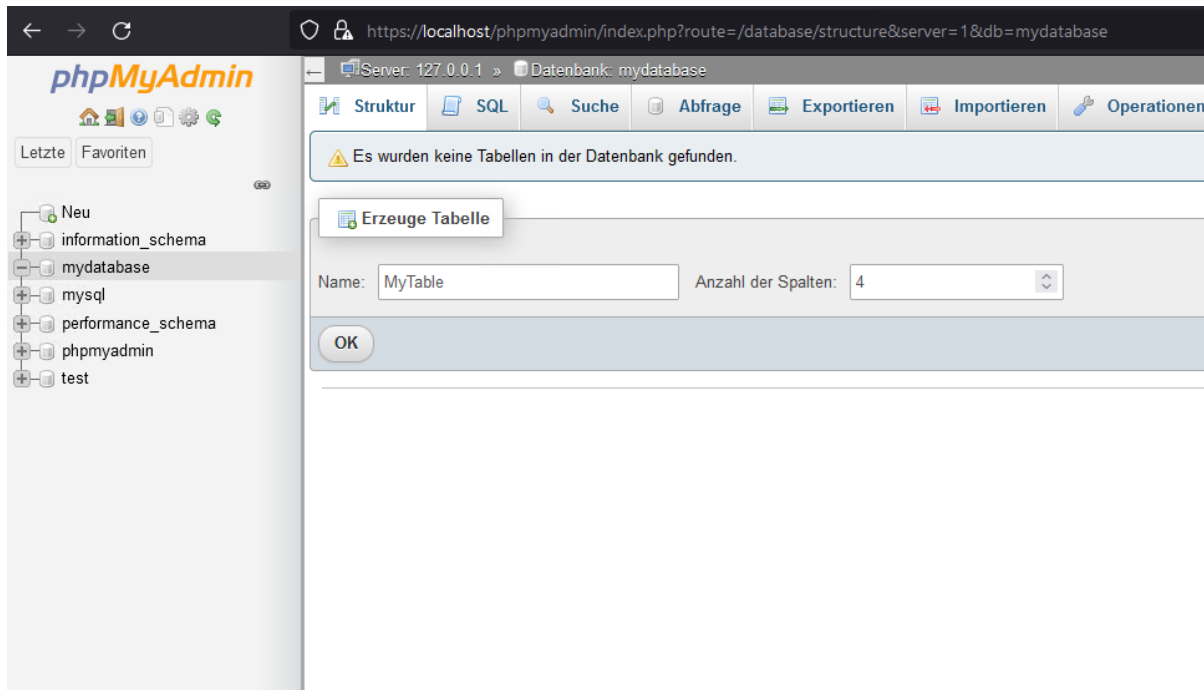
Starten Sie XAMPP, Apache und MySQL

Klicken Sie bei MySQL auf Admin

Es öffnet sich ein Browser mit PHP My Admin

Navigieren Sie zu den Datenbanken und erstellen Sie die Datenbank „MyDatabase“

Datenbank in PHP My Admin erstellen



Server: 127.0.0.1 » Datenbank: mydatabase

Struktur SQL Suche Abfrage Exportieren Importieren Operationen

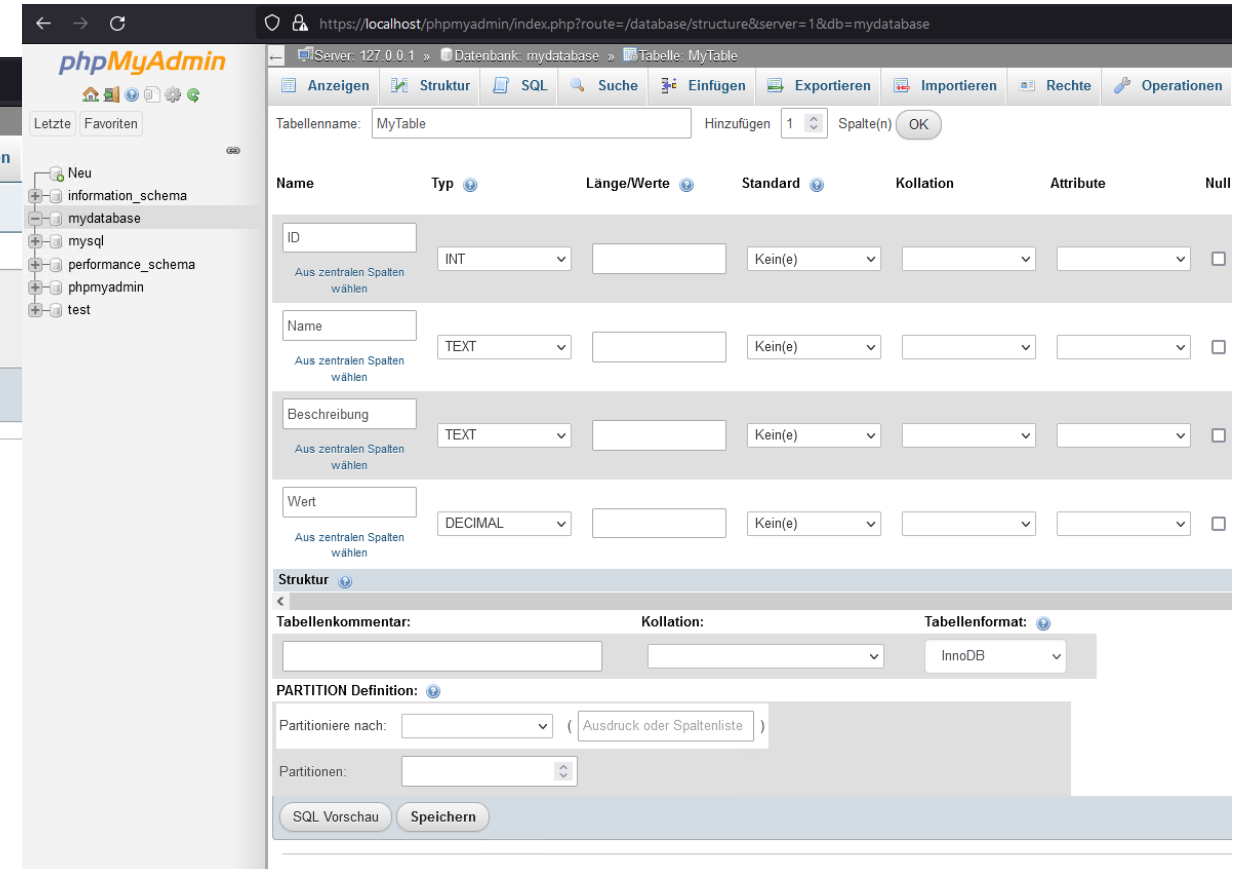
Es wurden keine Tabellen in der Datenbank gefunden.

Erzeuge Tabelle

Name: MyTable Anzahl der Spalten: 4

OK

Erzeugen Sie die Tabelle „MyTable“ mit 4 Spalten und klicken Sie auf OK



Server: 127.0.0.1 » Datenbank: mydatabase » Tabelle: MyTable

Anzeigen Struktur SQL Suche Einfügen Exportieren Importieren Rechte Operationen

Tabellenname: MyTable Hinzufügen 1 Spalte(n) OK

Name	Typ	Länge/Werte	Standard	Kollation	Attribute	Null
ID	INT		Kein(e)			☐
Name	TEXT		Kein(e)			☐
Beschreibung	TEXT		Kein(e)			☐
Wert	DECIMAL		Kein(e)			☐

Struktur

Tabellenkommentar: Kollation: Tabellenformat: InnoDB

PARTITION Definition:

Partitioniere nach: (Ausdruck oder Spaltenliste)

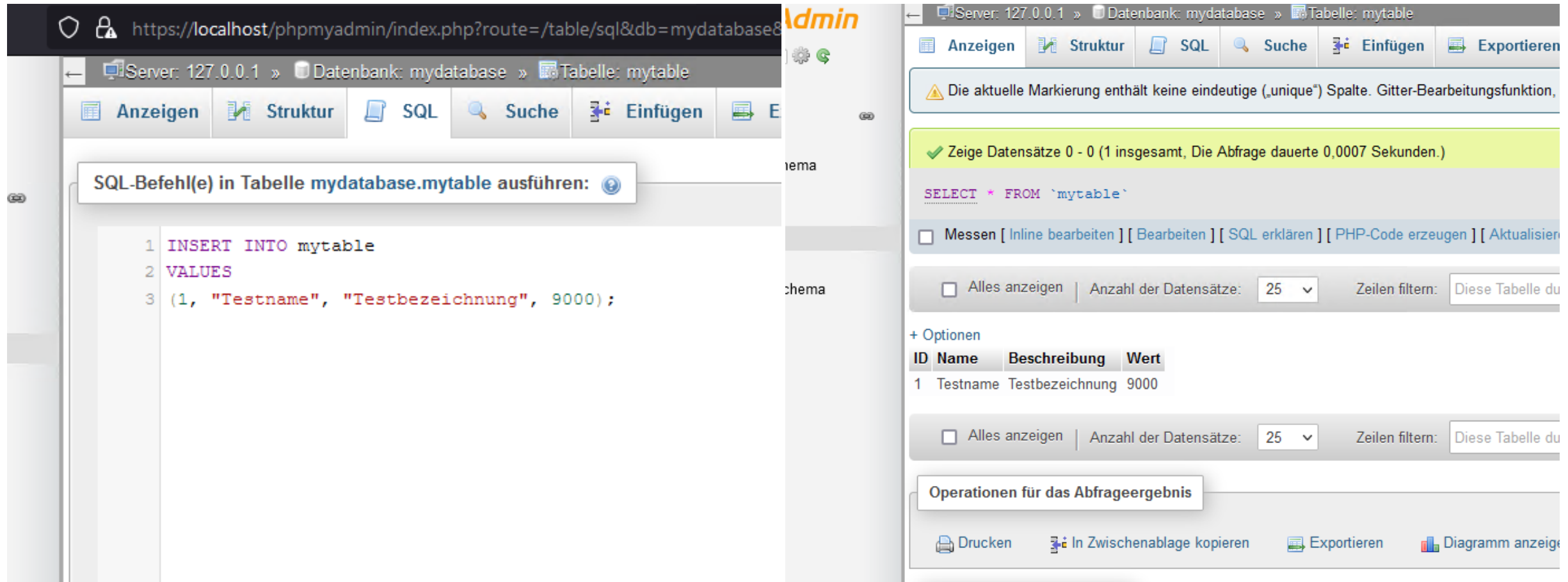
Partitionen:

SQL Vorschau Speichern

Erstellen Sie die Spalten „ID“, „Name“, „Beschreibung“, „Wert“ mit INT, TEXT, TEXT und DECIMAL entsprechend.

Klicken Sie anschließend unten auf Speichern

Datenbank in PHP My Admin erstellen



The left screenshot shows the phpMyAdmin interface for a database named 'mydatabase' and a table named 'mytable'. The SQL execution area contains the following command:

```
1 INSERT INTO mytable  
2 VALUES  
3 (1, "Testname", "Testbezeichnung", 9000);
```

The right screenshot shows the result of the query. It displays a single row in a table with the following data:

ID	Name	Beschreibung	Wert
1	Testname	Testbezeichnung	9000

Erzeugen Sie nun einen Eintrag in der Tabelle

03

PHP My Admin und IntelliJ

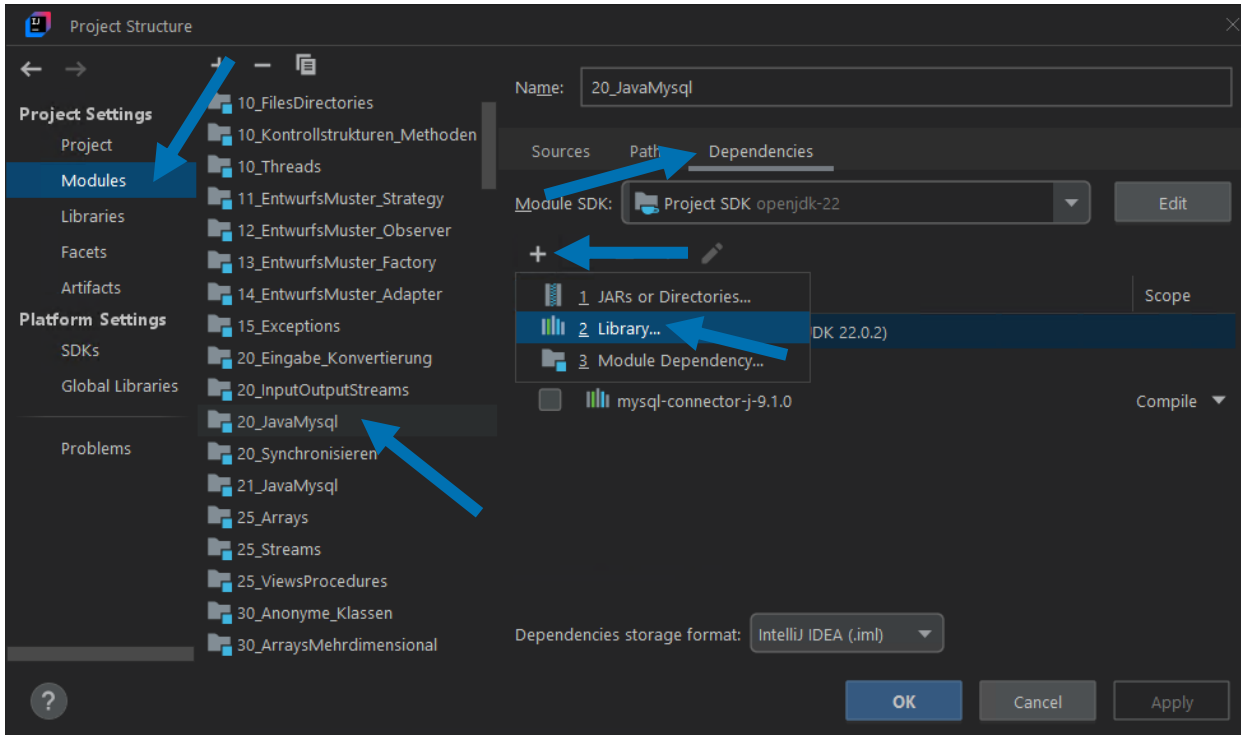
3.1 Datenbank in PHP My Admin erstellen

3.2 Datenbankabfrage in Java ←

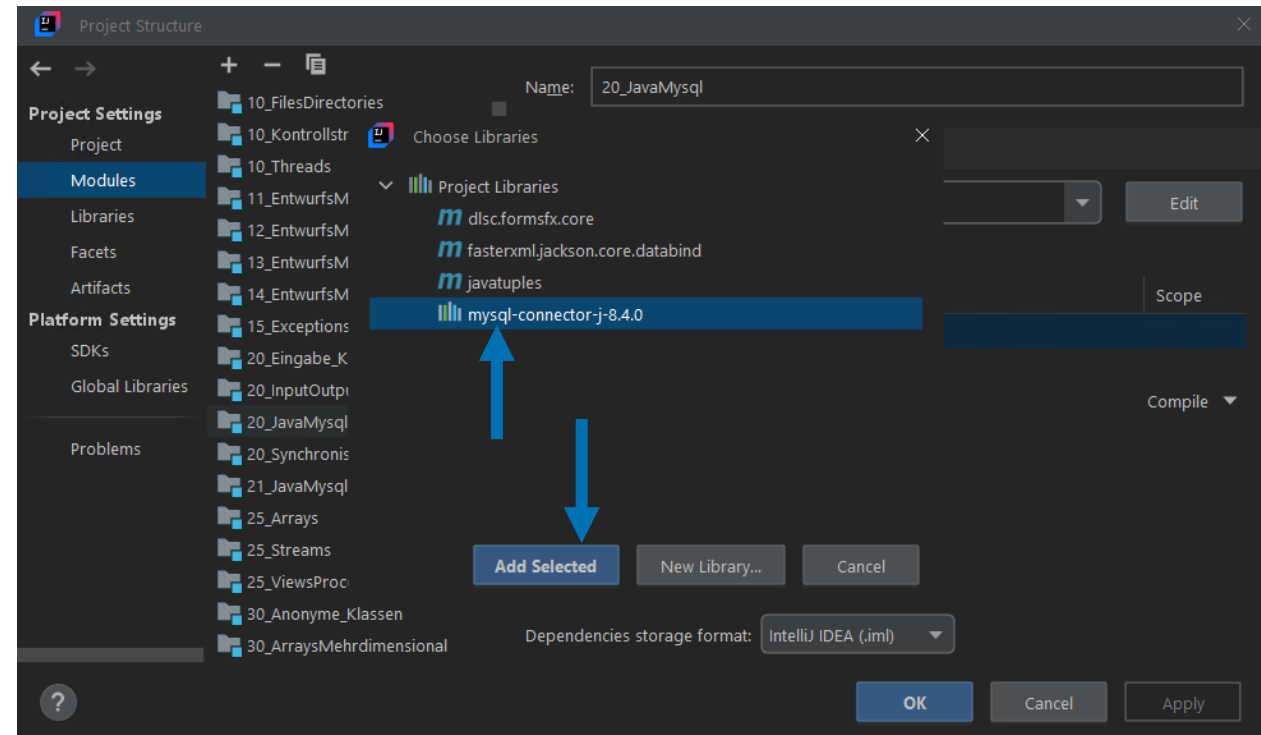
Zuerst erstellen wir eine Testdatenbank mit einer Tabelle und Beispieldaten in PHP My Admin und fragen diese Tabelle anschließend in Java ab.



Datenbankabfrage in Java



Bevor wir im Modul die Klassen des Connectors verwenden können, müssen wir die Library dem Modul als Dependency hinzufügen.



Über Project Structure -> Modules -> Modul auswählen -> Dependencies -> auf das + -> Library -> Library auswählen -> Add Selected

```
2
3 import java.sql.*;
4 |
5 import java.sql.Connection;
6 import java.sql.DriverManager;
7 import java.sql.SQLException;
8 import java.sql.ResultSet;
9 import java.sql.Statement;
10
11 public class Main {
12
```

Öffnen Sie wieder IntelliJ mit dem Projekt, welches den Verweis auf den Connector besitzt.

Fügen Sie dem Projekt die Import-Anweisung `java.sql.*` hinzu

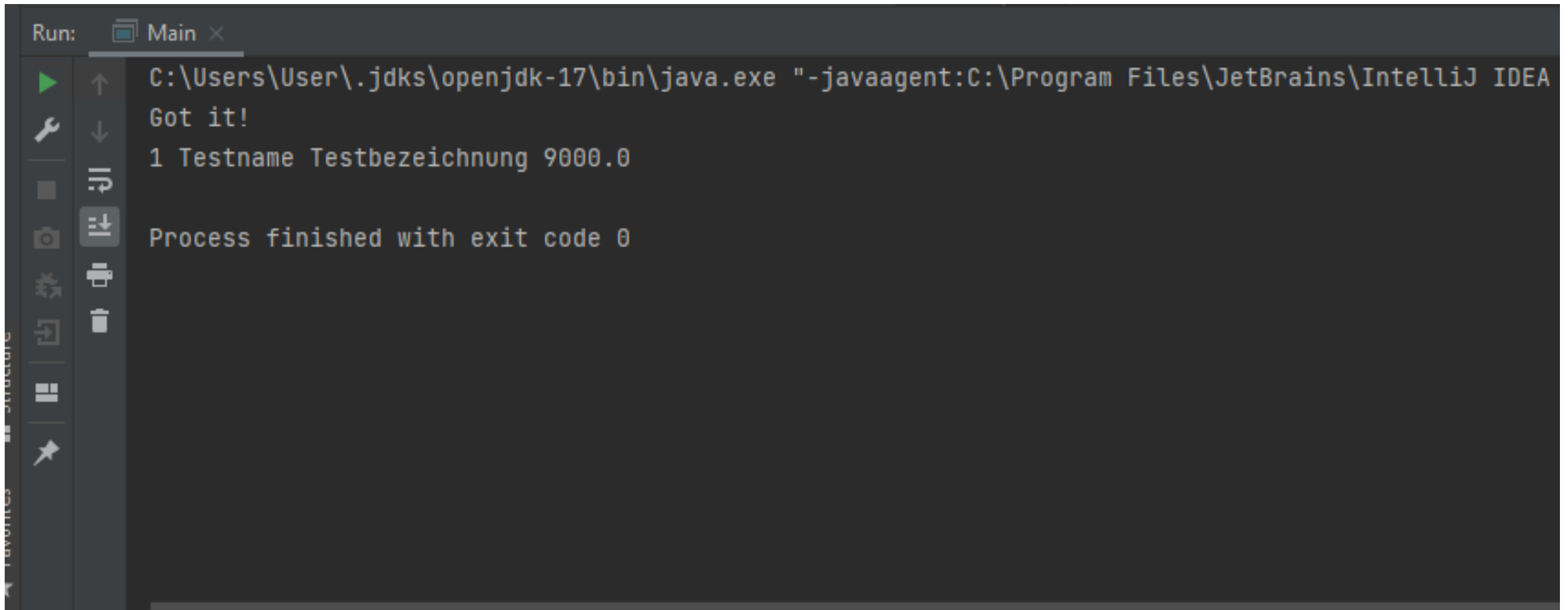
Für dieses Beispiel werden im Detail `.Connection`, `.DriverManager`, `.SQLException`, `.ResultSet` und `.Statement` benötigt.

```
12
13 public static void main(String[] args)
14 {
15     Connection connection = null;
16     try
17     {
18         String connectionString = "jdbc:mysql://localhost:3306/mydatabase";
19         connection = DriverManager.getConnection(connectionString, user: "root", password: "");
20
21         System.out.println("Got it!");
22
23         Statement statement = connection.createStatement();
24         ResultSet rs = statement.executeQuery( sql: "select * from mytable");
25         while(rs.next())
26         {
27             String sb = rs.getInt( columnLabel: "id") +
28                 " " + rs.getString( columnLabel: "name") +
29                 " " + rs.getString( columnLabel: "beschreibung") +
30                 " " + rs.getDouble( columnLabel: "wert");
31             System.out.println(sb);
32         }
33         connection.close();
34     }
35     catch (SQLException e)
36     {
37         System.out.println(e);
38     }
39 }
40
41 }
```

Durch eine Reihe von Anweisungen können wir nun einen Select Befehl auf der Datenbank ausführen und uns die Ergebnisse anzeigen lassen. Die Befehle sind auf der folgenden Folie genauer erläutert.

- `String connectionString` beinhaltet eine Zeichenfolge, mit der die Verbindung zur Datenbank aufgebaut werden kann. Verschiedene Datenbanken erfordern unterschiedliche Connection Strings
- `DriverManager.getConnection(...)` versucht nun, anhand des Connection Strings eine Verbindung zur Datenbank aufzubauen und gibt die Verbindung als Objekt zurück.
- `connection.createStatement()` erzeugt passend zu der Connection ein Statement-Objekt, über welches anschließend SQL Befehle an die Datenbank gesendet werden können.
- `statement.executeQuery(...)` führt die angegebene Query auf der Datenbank aus und gibt das Ergebnis der Abfrage als `ResultSet` zurück.
- mit `rs.next()` in einer Schleife kann Zeile für Zeile das Ergebnis durchlaufen werden.
- Die Getter-Methoden des `ResultSets` geben Zugriff auf die einzelnen Spalten über Index oder Spaltenname.
- Zum Schluss wird die Connection wieder geschlossen.
- Sollte es dabei zu einer Exception kommen, wird diese vom Try-Catch aufgefangen.

Datenbankabfrage in Java



```
Run: Main x
C:\Users\User\.jdk\openjdk-17\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA
Got it!
1 Testname Testbezeichnung 9000.0
Process finished with exit code 0
```

Wenn alles geklappt hat, sollten in der Ausgabe nun die Daten der abgefragten Tabelle erscheinen.

„Sie haben erfolgreich
eine SQL Abfrage über
Java ausgeführt.“



VIELEN DANK!

